

Topic: Polynomial Representation and Addition — Self-Learning

Q. Explain how a Singly Linked List can be used to represent a polynomial. Describe the algorithm to add two polynomials represented using linked lists with a suitable example.

📌 Definition to Remember

A polynomial (e.g., $3x^4 + 2x^2 + 5$) can be efficiently represented using a **Singly Linked List** where each node stores a non-zero term's **coefficient** and **exponent**. This is vastly superior to arrays for sparse polynomials, as it avoids wasting memory on zero-coefficient terms.

Polynomial Representation & Addition (Self-Learning)

Each node in the linked list represents one term and is divided into three parts:

1. **Coefficient:** The multiplier (e.g., the '3' in $3x^4$).
2. **Exponent:** The power (e.g., the '4' in $3x^4$).
3. **Next Pointer:** Links to the next term.

Structure in C:

```
struct PolyNode {
    int coefficient;
    int exponent;
    struct PolyNode *next;
};
```

(Nodes are kept sorted in descending order of their exponents to make addition easier).

Example Diagram for $5x^3 + 4x^1 + 2x^0$:

Head → [5|3|Next] → [4|1|Next] → [2|0|Next] → NULL

2. Polynomial Addition Algorithm

To add **Poly1** and **Poly2**, traverse both lists simultaneously using pointers **p1** and **p2**. Compare their exponents:

- **Case 1:** **p1->exponent > p2->exponent**
Append **p1** term to the Result list. Move **p1** forward.
- **Case 2:** **p1->exponent < p2->exponent**
Append **p2** term to the Result list. Move **p2** forward.
- **Case 3:** **p1->exponent == p2->exponent**
Add coefficients: **sum = p1->coef + p2->coef**.
If **sum != 0**, append a new node (**sum, exponent**) to Result.
Move both **p1** and **p2** forward.

After the loop, append any remaining terms from either **p1** or **p2** to the Result list.

3. Example Trace

- **Poly1:** $3x^2 + 5x + 2$
- **Poly2:** $4x^3 + 2x^2 + 1$
- **Resulting Steps:**
 1. Compare x^3 (Poly2) and x^2 (Poly1) → Add $4x^3$ to Result.
 2. Compare x^2 (Poly1) and x^2 (Poly2) → Add $(3+2)x^2 = 5x^2$ to Result.
 3. Compare x^1 (Poly1) and x^0 (Poly2) → Add $5x^1$ to Result.
 4. Compare x^0 (Poly1) and x^0 (Poly2) → Add $(2+1)x^0 = 3x^0$ to Result.
- **Final Result:** $4x^3 + 5x^2 + 5x + 3$

4. Advantages

- **Memory Efficiency:** Only non-zero terms are stored. (An array for $x^{100} + 1$ requires 101 slots; a linked list requires only 2 nodes).
- **Dynamic Size:** Easily accommodates the result of addition/multiplication without predefined limits.

★ Must-Write Points (for 10 marks)

1. Arrays waste memory for **sparse polynomials** (many zero terms). Linked Lists only store non-zero terms.
2. Node structure: `int coefficient` , `int exponent` , and `struct PolyNode *next` .
3. Terms are kept sorted in **descending order of exponents**.
4. **Addition Logic ($p1 > p2$):** Add $p1$ to result, advance $p1$.
5. **Addition Logic ($p1 < p2$):** Add $p2$ to result, advance $p2$.
6. **Addition Logic ($p1 == p2$):** Add coefficients together. If $sum \neq 0$, add to result. Advance both $p1$ and $p2$.
7. Append any leftover terms at the end of the lists.

⚡ Quick Recall

Poly LL → Node(coeff, exp, next) → Sorted by Exp (descending) → Add: if $exp1 > exp2$ (copy 1), if $exp1 < exp2$ (copy 2), if $exp1 == exp2$ (add coeffs) → Saves memory for sparse polys